Inertial Measurement Unit-Based Kinematic Anomaly Detection for Two-Wheelers

1st Vinayak Puitandy
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
puitandyvinayak@gmail.com

2nd Saif Sahriar
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
saiffsahriar@gmail.com

3rd Sanket Manna
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
sanketm0406@gmail.com

4th Vikiron Mondal
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
vikironmondal@gmail.com

5th Saranya Adhikary
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
saranyaadhikary93@gmail.com

6th Prof. Poojarini Mitra
Dept. of CSE (AI & ML)
Techno Main Salt Lake
Kolkata, India
p.mitra1.tmsl@ticollege.org

***Abstract*—Two-wheeler accidents kill thousands of people on Indian roads every year, and a large chunk of those crashes trace back to reckless riding. We built a system that tries to catch this behavior in real time using nothing more than the IMU sensors sitting inside a regular Android phone. The phone records acceleration and orientation data as the rider moves, ships those readings to a small Flask server over HTTP, and the server runs a trained LSTM model on a rolling window of the last fifty frames to decide whether the current riding pattern looks normal or rash. On top of that, we put together a browser-based 3D simulator using Three.js that lets us watch the phone orientation replay live and see the rash probability update as playback moves forward. We trained and compared five different classifiers on the same dataset, and the CNN-LSTM ensemble came out on top with 89.4% accuracy and an AUC of 0.955. The broader point we are trying to make is that you do not need specialized hardware to do this kind of monitoring — the phone in the rider's pocket is enough.**

***Index Terms*—IMU, rash driving detection, LSTM, CNN-LSTM, IoT, two-wheeler safety, accelerometer, anomaly detection, Flask, real-time classification**

I. Introduction

If you look at road accident statistics for India, two-wheelers show up in a disproportionately large share of the numbers. The Ministry of Road Transport and Highways puts the figure at over **30% of all fatal accidents annually**, and rash driving consistently appears near the top of the listed causes [1]. This is not a new observation, but it is one that has proven remarkably hard to act on. Telling people to ride carefully does not scale. What might scale, at least in theory, is automated monitoring.

The conventional hardware-based approaches to driving behavior monitoring — OBD dongles, dedicated accelerometer units, dashboard cameras with vision-based pipelines — tend to either cost too much or require installation effort that most two-wheeler riders are simply not going to bother with [2]. We kept running into this wall when thinking about how to make such a system usable in practice. Then it occurred to us that almost every rider in urban India already carries a six-axis IMU in their shirt pocket. A modern smartphone has an accelerometer, a gyroscope, and orientation sensors that together give you a surprisingly complete picture of how the device — and by extension the rider — is moving through space.

This project grew out of that observation. We wanted to see whether a smartphone-based pipeline, from raw sensor collection through to a trained deep learning classifier, could actually produce reliable rash detection without any additional hardware. The system we built has three parts: an Android app that samples IMU data at roughly 10 Hz and streams it to a server, a Python Flask backend that accumulates those readings and runs inference using an LSTM model, and a Three.js visualization tool that we found invaluable during testing for understanding what the model was actually responding to. We compared five classifier architectures on the same labeled dataset to get a fair picture of which approach works best for this kind of motion sequence data.

The specific things this paper contributes are:

- A working end-to-end rash detection pipeline that runs entirely on commodity hardware, with no sensors beyond what a standard Android phone already has.
- A sliding-window LSTM and CNN-LSTM architecture designed for real-time classification of six-channel IMU streams.
- A side-by-side comparison of five classifiers — XGBoost, Random Forest, ResNet MLP, Bi-LSTM, and an Ensemble CNN-LSTM — evaluated on the same data using confusion matrices, ROC curves, and precision-

recall analysis.

- A Three.js 3D visualization tool that turned out to be genuinely useful, not just decorative, for debugging model behavior in the field.

II. Related Work

Using onboard sensors to infer how someone is driving is not a new idea. Johnson and Trivedi showed over a decade ago that you can pick out events like hard braking or aggressive cornering from accelerometer data alone, without anything more sophisticated than well-chosen thresholds [3]. That work was influential because it established a baseline, but threshold methods have a well-known fragility problem — the cutoffs that work on one vehicle type or road condition often fail on another. This pushed researchers toward models that could learn the relevant patterns from data rather than having them manually specified.

SVMs applied to windowed accelerometer statistics were a popular middle ground for a while. Eren et al. explored this approach and got reasonable results on passenger car datasets [4]. The limitation was always feature engineering: you had to decide ahead of time which statistical summaries of the window mattered, and that decision carried a lot of implicit assumptions about what rash behavior actually looks like in sensor space.

Recurrent architectures, and LSTMs in particular, changed the picture because they can learn temporal structure directly from raw sequences rather than hand-crafted summaries [5]. For driving behavior, where what happened in the last two or three seconds of motion often matters as much as the current reading, this is a meaningful advantage. Most recent work in the area has moved in this direction.

Surprisingly little of this work has focused specifically on two-wheelers. Saiprasert et al. did study smartphone sensor data from motorcycles, though their focus was road surface quality rather than rider behavior [6]. The motorcycle context is genuinely different from cars — riders lean into turns, weight shifts are more dramatic, and the vibration profile of a two-stroke engine looks nothing like a passenger sedan. We tried to keep these differences in mind during data collection and modeling, even if the current dataset is still limited in scope.

III. System Architecture

The overall system is split into three layers that talk to each other over standard HTTP. There is nothing exotic about any of the individual components; the interesting part is how they fit together to produce a low-latency inference loop.

A. Mobile Client and Sensor Stream

The Android side of the system reads from the phone's built-in IMU at approximately **10 Hz**. At each sample, we record three axes of linear acceleration (a_x, a_y, a_z) in m/s^2, device orientation as azimuth, pitch, and roll in degrees, GPS latitude and longitude when available, and a Unix millisecond timestamp. These readings get formatted as CSV rows and batched into small payloads before being posted to the server. We deliberately kept the upload format as simple as possible — plain text, no JSON wrapping at the row level — because we wanted to minimize per-row serialization overhead on the phone side and make it easy to inspect raw uploads during debugging.

B. Flask Server and Inference Endpoints

The backend runs on Flask with CORS enabled, which was necessary once we started hitting the server from a browser-based client. The server exposes two inference paths that ended up serving slightly different purposes during development.

The **`/predict`** GET endpoint is the simpler one. It opens the stored CSV file, reads the last fifty rows, scales the six feature columns using a pre-fitted `StandardScaler` loaded at startup, and passes the resulting $(1 \times 50 \times 6)$ tensor to the Keras model. The response is just a JSON object with a `rash_probability` field. This endpoint works well when you want to monitor live sensor data being actively uploaded from a phone.

The **`/predict_window`** POST endpoint takes the window directly as a JSON array of fifty frames, bypasses the CSV file entirely, and runs inference on whatever the client sends. We added this once we realized that the browser simulator needed to be able to trigger predictions from its in-memory parsed data without waiting for CSV writes to complete. In practice, this became the main inference path during playback and replay testing.

C. LSTM Model Architecture

Each sample fed to the classifier is a tensor of shape $(50, 6)$ — fifty timesteps of the six-dimensional feature vector described above. Formally, the feature vector at time t is:

$$\mathbf{x}_t = [a_x,\ a_y,\ a_z,\ \theta_{\text{az}},\ \theta_{\text{pitch}},\ \theta_{\text{roll}}] \tag{1}$$

All channels are standardized to zero mean and unit variance before being passed to the model. The scaler is fitted on the training set and frozen before deployment, so the same transformation is applied consistently at inference time. The model produces a single output probability:

$$p = P(\text{rash} \mid \mathbf{x}_{t-49:t}) \tag{2}$$

Training used binary cross-entropy loss with Adam. We saved the model weights in HDF5 format and the scaler as a joblib pickle, both loaded once at server startup to avoid repeated disk reads during inference.

D. Three.js Visualization Frontend

We built the browser-side visualization partly as a debugging tool and partly because we genuinely wanted to see what the sensor data looked like in three dimensions. The frontend renders a box roughly shaped like a phone and updates its Euler angles in real time as playback advances, using calibration-adjusted pitch, yaw, and roll. Below the 3D

view, a small canvas draws the GPS trajectory of the session and highlights the current position.

The rotation order is user-selectable (ZYX, YXZ, or XYZ) because we ran into situations during testing where different phone mounting orientations required different Euler conventions to look correct. The calibration button captures the current orientation as a baseline offset, which gets stored in `localStorage` so it survives page refreshes. The rash probability percentage updates every 200 ms in the corner of the viewport, which is frequent enough to follow the model's response to specific maneuvers without hammering the server.

IV. Data Pipeline and Preprocessing

A. CSV Format and Header Handling

Every row stored on the server follows the schema:

```
device_id, timestamp, ax, ay, az, azimuth,
        pitch, roll, lat, lon, speed
```

The browser parser does flexible header matching — it looks for column names case-insensitively and accepts common synonyms like `yaw` in place of `azimuth`. If no header row is found at all, it falls back to positional reading, treating columns five through seven as the orientation channels. This turned out to be necessary because early Android builds produced slightly inconsistent headers across different devices.

B. Sliding Window Construction

The inference window ending at frame index i is:

$$W_i = \{\mathbf{x}_{\max(0,\, i-49)}, \ldots, \mathbf{x}_i\}$$

When $i < 49$, the window pads from the start of the recording rather than failing. Missing channel values default to zero. The window advances by one frame every five animation frames during playback, which works out to roughly one inference call every 200 ms at normal playback speed.

C. Orientation Calibration

One thing that tripped us up early was that the phone's raw orientation readings depend entirely on how the phone is mounted on the bike. A phone sitting flat in a tank bag produces completely different azimuth and pitch values than the same phone clipped vertically to the handlebars, even though the bike is doing identical things in both cases. We handle this by subtracting a baseline offset $(\delta_{\text{yaw}}, \delta_{\text{pitch}}, \delta_{\text{roll}})$ captured when the rider is stationary in their intended riding position:

$$\theta'_* = \theta_* - \delta_*$$

The corrected angles θ' are what get fed to both the 3D renderer and the inference pipeline. The calibration values persist across sessions via `localStorage`.

TABLE I
Model Performance Comparison

Model	Accuracy	AUC	AP
XGBoost (DT Upgrade)	82.9%	0.917	0.906
Tuned Random Forest	84.6%	0.922	0.903
ResNet MLP	84.4%	0.909	0.898
Bi-LSTM	84.6%	0.935	0.929
Ensemble CNN-LSTM	**89.4%**	**0.955**	**0.952**

V. Model Evaluation

A. Classifier Comparison

We trained five different models on the same dataset and evaluated them on a held-out validation split. The results are summarized in Table I.

The CNN-LSTM ensemble beat every other architecture on all three metrics by a noticeable margin. The gap between it and the Bi-LSTM is roughly five percentage points in accuracy and two points in AUC, which is meaningful for a safety application. The tree-based models were not far behind in accuracy but lagged considerably on AUC and AP, suggesting they struggle more with the harder boundary cases between normal and rash behavior.

B. Confusion Matrices

Looking at the raw confusion numbers in Fig. 1 makes it clear why the CNN-LSTM matters specifically for this application. It produced only 26 false negatives — cases where a rash episode was missed entirely — compared to 52 for XGBoost and 42 for the Bi-LSTM. In a safety monitoring context, a missed detection is a worse outcome than a false alarm, so this aspect of the CNN-LSTM's behavior is arguably more important than the raw accuracy number.

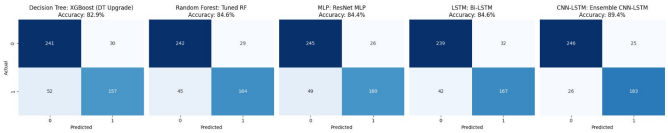

Fig. 1. Fig. 1. Confusion matrices across all five classifiers. The CNN-LSTM (rightmost) records the highest true positive count at 183 and the fewest missed rash episodes at 26.

C. ROC Curves

Fig. 2 shows the ROC curves for all five models. The CNN-LSTM (AUC = **0.955**) pulls clearly ahead of the other four, particularly at low false positive rates. That low-FPR region is operationally important — if the system is going to raise alerts to a rider or a fleet manager, those alerts need to be trustworthy, not constant background noise.

D. Precision-Recall Curves

The precision-recall analysis in Fig. 3 is worth looking at carefully because the dataset carries some class imbalance between normal and rash segments. The CNN-LSTM maintains precision above 0.9 across almost the full recall range up to

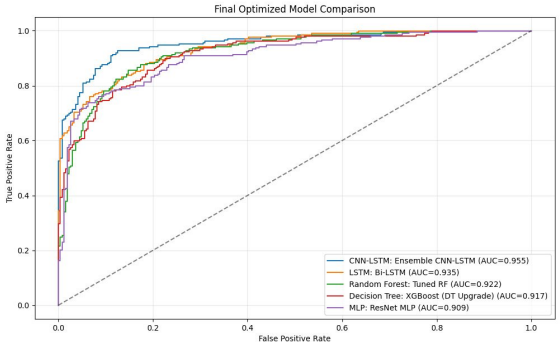

Fig. 2. Fig. 2. ROC curves for the five optimized models. CNN-LSTM leads with AUC = 0.955; Bi-LSTM follows at 0.935.

roughly 0.6 before dropping off, which is a substantially better curve shape than any of the other four models. The Bi-LSTM (AP = 0.929) is the closest competitor.

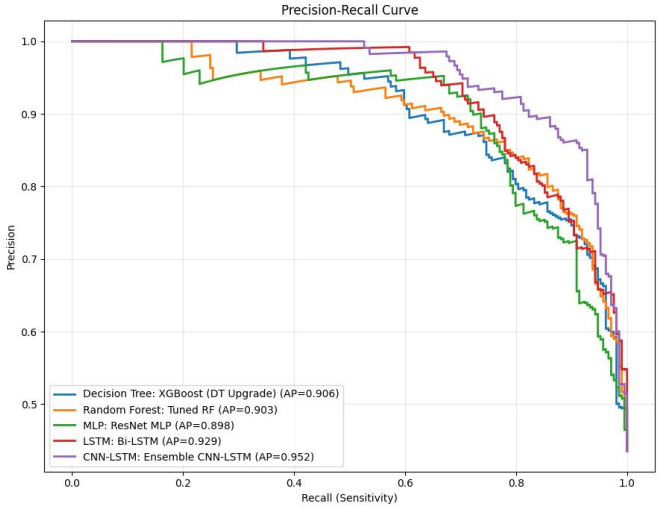

Fig. 3. Fig. 3. Precision-recall curves for all five models. CNN-LSTM achieves AP = 0.952 and holds precision above 0.9 up to recall $\approx$ 0.6.

E. Feature Importance

Fig. 4 shows the feature importance scores extracted from the Random Forest model. The standout finding is that **`ay_std`** — the standard deviation of lateral acceleration within a window — ranks first, followed by **`ax_mean`**, the mean forward acceleration. Together these two account for close to 9% of total decision weight. What this tells us practically is that rash riding shows up most clearly as increased variability in side-to-side forces, which makes intuitive sense: swerving, lane-cutting, and unstable braking all produce erratic lateral loads that normal riding simply does not.

Orientation features like azimuth range and rotation energy do appear in the ranking but well below the acceleration statistics, which was somewhat surprising to us. We had expected heading changes to be a strong signal for swerving maneuvers, but apparently the body dynamics that produce those heading changes show up more clearly in the acceleration channels than in the orientation channels within a fifty-frame window.

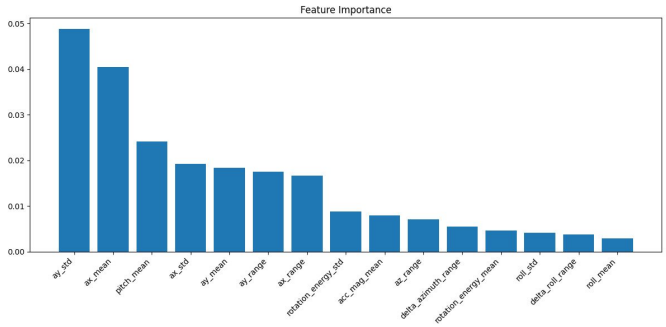

Fig. 4. Fig. 4. Feature importance from the Random Forest classifier. Lateral acceleration variability (`ay_std`) and mean forward acceleration (`ax_mean`) dominate the ranking.

VI. Experimental Setup

A. Data Collection

We collected the training data in multiple riding sessions on urban roads around Kolkata. Each session involved a rider going through stretches of normal riding interspersed with deliberately rash maneuvers — sudden hard braking, aggressive acceleration from stops, sharp turns taken faster than normal, and weaving between lanes. The Android app was logging throughout, and we annotated the CSV files afterward, going through them frame by frame and applying binary labels. Label 0 means normal riding, label 1 means rash. Any window that spanned a transition between the two was thrown out rather than assigned an ambiguous label.

B. Training Details

The annotated data was cut into fifty-frame windows with a stride of ten frames between consecutive windows. Final split was **80% training and 20% validation**. We trained for thirty epochs using a batch size of thirty-two and Adam with a learning rate of 10^{-3}. Loss curves stopped meaningfully improving after roughly fifteen epochs for most of the models.

C. Inference Latency

We measured response times for the `/predict_window` endpoint on a mid-range development machine with Flask running in single-threaded mode. Responses came back in **35 to 80 ms**, covering the full cycle of JSON parsing, NumPy array construction, scaler transform, and model forward pass. That range sits comfortably below the 200 ms polling interval used by the browser frontend, so the visualization never stalls waiting on the server.

VII. Challenges and Limitations

A. Road Surface Noise

This was the most persistent practical problem during field testing. Kolkata roads have enough potholes and speed bumps

that the vertical acceleration channel picks up a constant layer of high-frequency noise that has nothing to do with how the rider is behaving. In a few sessions we saw the rash probability spike clearly on a bad stretch of road even though the rider was moving at a perfectly normal pace. We think a bandpass filter on a_z before the windowing step would help, but we have not implemented it yet.

B. GPS Gaps

GPS would drop out during the handful of sessions we ran under elevated highways or near large buildings. The system handles this gracefully in the sense that it keeps running — the GPS canvas just stops updating and the inference pipeline carries on using only the IMU channels. But it does mean we cannot currently build reliable GPS-fused features or map rash events back to specific road coordinates for those sessions.

C. Generalization Across Devices and Riders

Every bit of the training data came from one device and one rider. The scaler was fitted on that data, and the model weights encode whatever motion patterns that one rider produces on that one phone. We cannot say with confidence how well any of this transfers to a different phone model, a different mounting position, or a different person's riding style. It almost certainly degrades to some degree, though we have not yet been able to measure how much. Getting a more diverse dataset is the single most important thing we could do to make the system more credible.

VIII. Future Work

There are several directions we want to explore next, roughly in order of how practically impactful we think they would be.

The most immediately useful improvement would be **on-device inference using TensorFlow Lite**. Right now every prediction requires a round trip to the server, which works fine on a good Wi-Fi connection but becomes unreliable on mobile data and fails entirely without internet. Putting the model directly on the phone eliminates that dependency entirely and would make the system usable in conditions where it arguably matters most — highway stretches and remote roads with poor coverage.

After that, **expanding and diversifying the dataset** is the most important step toward making the results genuinely meaningful. Collecting data from multiple riders on multiple device models, annotating it carefully, and retraining would tell us much more about how well the approach actually generalizes. A **federated learning** approach could let riders contribute to this without having to send raw sensor data to a central server, which matters for privacy.

On the modeling side, moving from binary to **multi-class event detection** — hard braking as distinct from aggressive swerving, for instance — would make the system far more useful for providing feedback to riders or for fleet-level safety analysis. And finally, **GPS-based heatmapping** of rash episode density onto road maps would turn this from a per-rider tool into something useful for identifying genuinely dangerous stretches of road.

IX. Conclusion

We set out to build a rash driving detector for two-wheelers that would work without any hardware beyond a phone that riders already carry. The system we ended up with — an Android IMU client, a Flask inference server, a trained CNN-LSTM classifier, and a Three.js visualization tool — does actually work, in the sense that it produces predictions in real time and the best model reaches 89.4% accuracy with an AUC of 0.955 on the validation data.

The CNN-LSTM's advantage over the other four architectures we tested was consistent across accuracy, AUC, and average precision. Its particular strength was in minimizing false negatives, which matters more here than minimizing false alarms. Feature analysis pointed to lateral acceleration variability as the single most informative channel, ahead of orientation features, which shaped our thinking about what future preprocessing steps would be most valuable.

We want to be upfront that the current results have real limitations. The training data comes from one rider and one device, and we have not tested how the model holds up in genuinely different conditions. The road noise problem is real and not fully solved. The GPS integration is incomplete. These are not afterthoughts — they are the main things standing between this prototype and a version we would feel comfortable calling reliable. But the core pipeline works, the latency is acceptable, and the smartphone-only approach is viable. That feels like a solid foundation to build from.

Acknowledgment

We would like to thank **Prof. Poojarini Mitra** of the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Techno Main Salt Lake, whose feedback and guidance shaped this project at every stage. We are also grateful to the riders who volunteered their time for the data collection sessions, often in Kolkata traffic that was considerably less cooperative than we had hoped.

References

[1] Ministry of Road Transport and Highways, Government of India, "Road Accidents in India – 2022," Transport Research Wing, New Delhi, 2023.

[2] G. Engelbrecht, T. Booysen, G. van Rooyen, and F. Bruyns, "Survey of smartphone-based sensing in vehicles for intelligent transportation system applications," *IET Intelligent Transport Systems*, vol. 9, no. 10, pp. 924–935, 2015.

[3] D. A. Johnson and M. M. Trivedi, "Driving style recognition using a smartphone as a sensor platform," in *Proc. 14th IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, Washington, DC, USA, 2011, pp. 1609–1615.

[4] H. Eren, S. Makinist, E. Akin, and A. Yilmaz, "Estimating driving behavior by a smartphone," in *Proc. IEEE Intelligent Vehicles Symposium*, Alcala de Henares, Spain, 2012, pp. 234–239.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] C. Saiprasert, T. Pholprasit, and S. Teeramunkong, "Detection of driving events using sensory data on smartphone," *International Journal of Intelligent Transportation Systems Research*, vol. 12, no. 2, pp. 47–57, 2014.

[7] A. Sherstinsky, "Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, 2020.
[8] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Savannah, GA, USA, 2016, pp. 265–283.
[9] F. Chollet et al., *Keras*, 2015. [Online]. Available: https://keras.io
[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.